

# AI FAQ Assistant

Secure FAQ management, content drafting, and source-backed answer retrieval.

Node.js + Express

MongoDB + Mongoose

JWT + bcrypt

Configurable AI integration

|              |             |
|--------------|-------------|
| Student name | [YOUR NAME] |
|--------------|-------------|

|                        |                    |
|------------------------|--------------------|
| Register / roll number | [YOUR ROLL NUMBER] |
|------------------------|--------------------|

|            |                   |
|------------|-------------------|
| Department | [YOUR DEPARTMENT] |
|------------|-------------------|

|             |                |
|-------------|----------------|
| Institution | [YOUR COLLEGE] |
|-------------|----------------|

|               |              |
|---------------|--------------|
| Project guide | [YOUR GUIDE] |
|---------------|--------------|

|               |                      |
|---------------|----------------------|
| Academic year | [YOUR ACADEMIC YEAR] |
|---------------|----------------------|

Prepared from the implemented project and its verification results. Complete the student and institution details before submission.

# Project overview

## Abstract

AI FAQ Assistant is a web application and REST API for creating, organizing, and retrieving frequently asked questions. It combines a Node.js and Express backend with MongoDB through Mongoose, JWT-based authentication, bcrypt password hashing, and a responsive browser interface. Users maintain separate knowledge bases, prepare FAQ drafts from source text, review drafts before saving, and test answer retrieval.

The implementation supports two explicit operating modes. When an OpenAI API key is configured, structured generation produces FAQ drafts and text embeddings support semantic retrieval. Without a key, an extractive drafting and keyword/synonym search mode allows an offline demonstration after dependencies are installed. A persistent JSON adapter provides an alternative to MongoDB for rapid local setup. The interface identifies the active mode. This report describes implemented functionality and distinguishes provider contract tests from live AI evaluation.

## 1. Introduction

Organizations repeatedly answer questions about delivery, billing, account access, and support. Answers spread across documents or individual conversations are difficult to maintain. A central FAQ knowledge base can reduce repeated manual work and make answers more consistent.

This project separates knowledge preparation from answer retrieval. A user can create an answer manually or derive draft questions and answers from source material. Every generated draft must be reviewed before it becomes searchable. The test assistant then retrieves a saved answer and displays the source question.

## 2. Problem statement

Traditional static FAQ pages require manual writing and exact browsing. Users may phrase the same question differently, making simple literal lookup insufficient. Teams also need access controls to prevent one user's content from being changed by another.

The project addresses these needs through authenticated FAQ management, a structured drafting workflow, owner-scoped storage, and two retrieval implementations suitable for demonstrating the difference between lexical and semantic matching.

# Scope and requirements

## 3. Objectives

- Build a working REST API using Node.js and Express.
- Authenticate users and protect private knowledge-base operations.
- Store and validate FAQ documents using MongoDB and Mongoose.
- Generate reviewable FAQ drafts from source content.
- Retrieve relevant saved answers and expose their source.
- Provide a usable desktop/mobile interface for demonstration.
- Validate authentication, isolation, persistence, and core browser workflows.

## 4. Scope

The implemented system includes registration, login, logout, per-user FAQ creation, reading, updating, deletion, category filtering, text filtering, export, source-text drafting, and answer retrieval. The assistant is a retrieval interface: it returns an existing answer rather than composing a free-form conversation.

Not implemented: email verification, password recovery, shared team permissions, document-file parsing, multilingual evaluation, human-agent handoff, analytics, deployment, and a persistent vector database. Pasted text is the supported generation input. Sample customer policies are fictional demonstration data.

## 5. Requirements

Software: Node.js 22 or newer, npm, a modern browser, and optionally MongoDB and an OpenAI API account. Windows launchers are provided. The local MongoDB launcher can start an actual database process automatically. Its first run on a new computer requires a binary download.

Functional requirements include authenticated access, validated FAQ operations, source-grounded drafts, explicit draft approval, relevant retrieval, a no-answer result, and data export. Nonfunctional requirements include clear errors, password protection, owner isolation, responsive layout, and predictable local startup.

Operating modes are explicit: MongoDB or local JSON storage; provider-backed AI or the no-key extractive and keyword demo. These modes are not interchangeable claims.

# Architecture and security

## 6. System architecture

Browser interface > Express REST API > authentication and validation > service modules > storage adapter.

The storage adapter selects MongoDB or local JSON at startup. The AI service selects the configured provider or the labelled offline demo algorithm. The browser never receives the AI key. User input passes through Zod validation before reaching application services. Central error handling returns a safe message and request identifier.

Main source files:

- src/app.js: routes and security middleware.
- src/store.js: User and FAQ schemas and persistence adapters.
- src/ai.js: generation, embedding retrieval, and demo algorithms.
- src/server.js: configuration and startup.
- public/app.js: UI state and API interactions.

## 7. Database design

The User entity contains id, name, normalized email, passwordHash, and createdAt. Email has a unique index in MongoDB. A FAQ contains id, owner, question, answer, category, source, createdAt, and updatedAt. The owner field is indexed.

One user owns many FAQ records. All read/update/delete operations include the current user's identity. IDs use UUID strings consistently across database implementations. FAQ question and answer lengths are constrained both at the API and database levels.

Local JSON data and MongoDB data are independent stores. Switching modes does not migrate accounts. The JSON adapter serializes writes and replaces a temporary file to avoid overlapping writes within one process.

## 8. Authentication and security

Registration validates the input, normalizes the email address, hashes the password with bcrypt at cost 12, and saves the user. Login verifies the hash and issues a JWT with subject, issuer, audience, and an eight-hour expiry. Browser sessions use HttpOnly, SameSite=Strict cookies. HTTPS production mode adds Secure.

Protected routes validate the token, load the user, and scope FAQ operations to that user's id. Zod rejects extra fields and malformed data. Helmet supplies security headers. Rate limiting covers general API traffic, authentication failures, and AI requests. Cross-site browser mutations are blocked. Frontend rendering escapes stored text.

Logout expires the browser cookie. Because tokens are stateless, previously issued bearer tokens are not revoked until expiry. Adding revocation and account recovery is future work. The project includes defensive controls but is not a substitute for a production security audit.

# Drafting and retrieval

## 9. FAQ generation

In provider mode, the source text is sent to the Responses API with instructions to use only supplied facts and treat embedded instructions as untrusted source data. Structured output constrains the result to an array of question, answer, and category objects. The server validates that response before returning drafts. The user reviews and saves each draft.

In demo mode, explicit Q/A pairs are extracted where present. Otherwise, sentence matching and topic templates produce drafts. Answers are copied from the source sentences. This mode demonstrates the workflow but is not a language model. A user should review both modes for relevance and correctness.

## 10. Search and answer retrieval

Semantic mode embeds the query and FAQ texts, ranks cosine similarity, and returns the strongest candidates. Embeddings are cached in memory to reduce repeated provider requests. A top-score threshold decides whether to return a saved answer or a no-match message.

Demo mode tokenizes text, removes stopwords, normalizes a small set of synonyms, and computes query-term overlap. Matches in question text receive additional weight. This helps demonstrate paraphrases such as "money back" and "refund" but does not provide general semantic understanding.

Similarity scores are not confidence probabilities. The thresholds are heuristic and have not been calibrated on a labelled evaluation set. The retrieved answer is returned verbatim with its source to make the result inspectable.

Drafting workflow: paste source text > create drafts > review and edit > save to the knowledge base.

Retrieval workflow: submit question > rank stored FAQs > apply a match threshold > return the saved answer with its source, or a no-match response.

# Verification and evaluation

## 11. Testing and results

The automated Node test run passed 20 tests/subtests. Coverage includes protected routes, registration, duplicate accounts, login failures, cookie authentication, forged tokens, request validation, owner isolation, retrieval, no-match handling, draft generation, persistence across reopening, cross-site rejection, malformed JSON, repeat-safe sample loading, deletion, logout, static serving, and AI provider contracts.

The MongoDB integration script passed using a real temporary MongoDB process. It validated connection, schemas, unique email handling, CRUD operations, and owner isolation.

Browser testing passed on desktop and a 390-pixel mobile viewport. It covered sample login, creating/editing/deleting an FAQ, filtering, draft generation, draft review, source-backed answers, unknown questions, session reload, and logout. No JavaScript runtime errors or horizontal mobile overflow were observed.

Live OpenAI API calls were not made because no API key was supplied. Mocked provider tests validate request/response handling; they do not establish real model quality. No numerical accuracy, latency, or load-capacity claims are made. Screenshots are included in docs/screenshots.

## 12. Limitations and future work

The system scans FAQ records in memory and caches vectors only for the lifetime of the process. A production-scale version should add a persistent vector index, batch-size controls, pagination, monitoring, backups, and evaluation datasets. Local JSON is a single-process demo option.

Future extensions include password recovery, email verification, audit logging, document upload and parsing, human escalation, multilingual support, and quality evaluation using labelled questions. A live AI demonstration requires a valid provider key and network access.

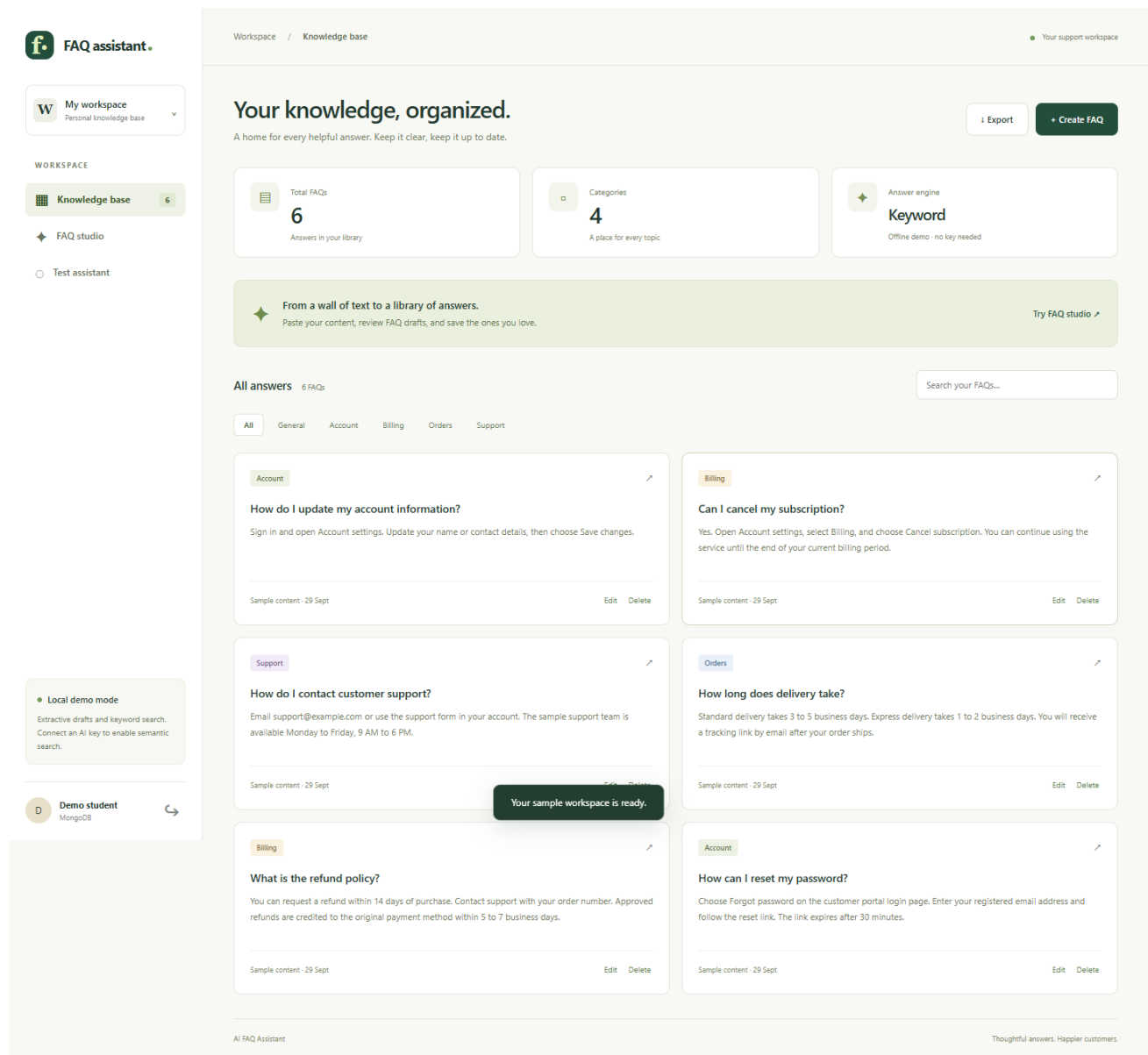
## 13. Conclusion

The project implements a complete FAQ-management and retrieval workflow with authentication, user isolation, validated persistence, a review stage, and a responsive UI. It provides MongoDB support and a configurable real AI integration while preserving a runnable no-key demo. Its tests establish the implemented application behavior; AI quality remains a separate evaluation task.

# Application screens

The screenshots below show the implemented browser interface using sample content.

**Figure 1. Knowledge base**

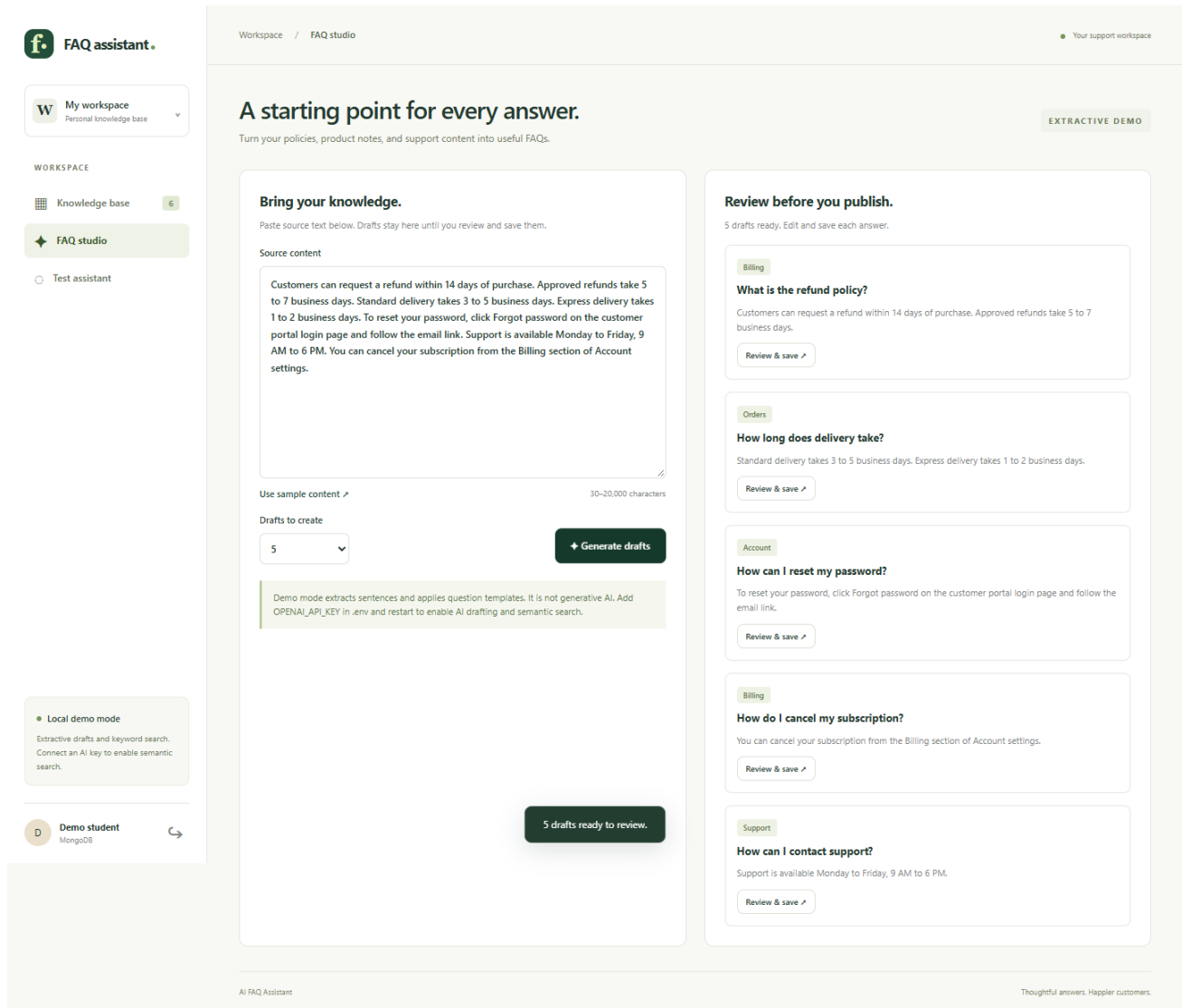


FAQ cards, categories, filters, creation, editing, deletion, and JSON export are available from the knowledge-base workspace. Sample business policies are demonstration content.

# FAQ studio

A review stage separates draft preparation from saved knowledge.

**Figure 2. Draft generation and review**



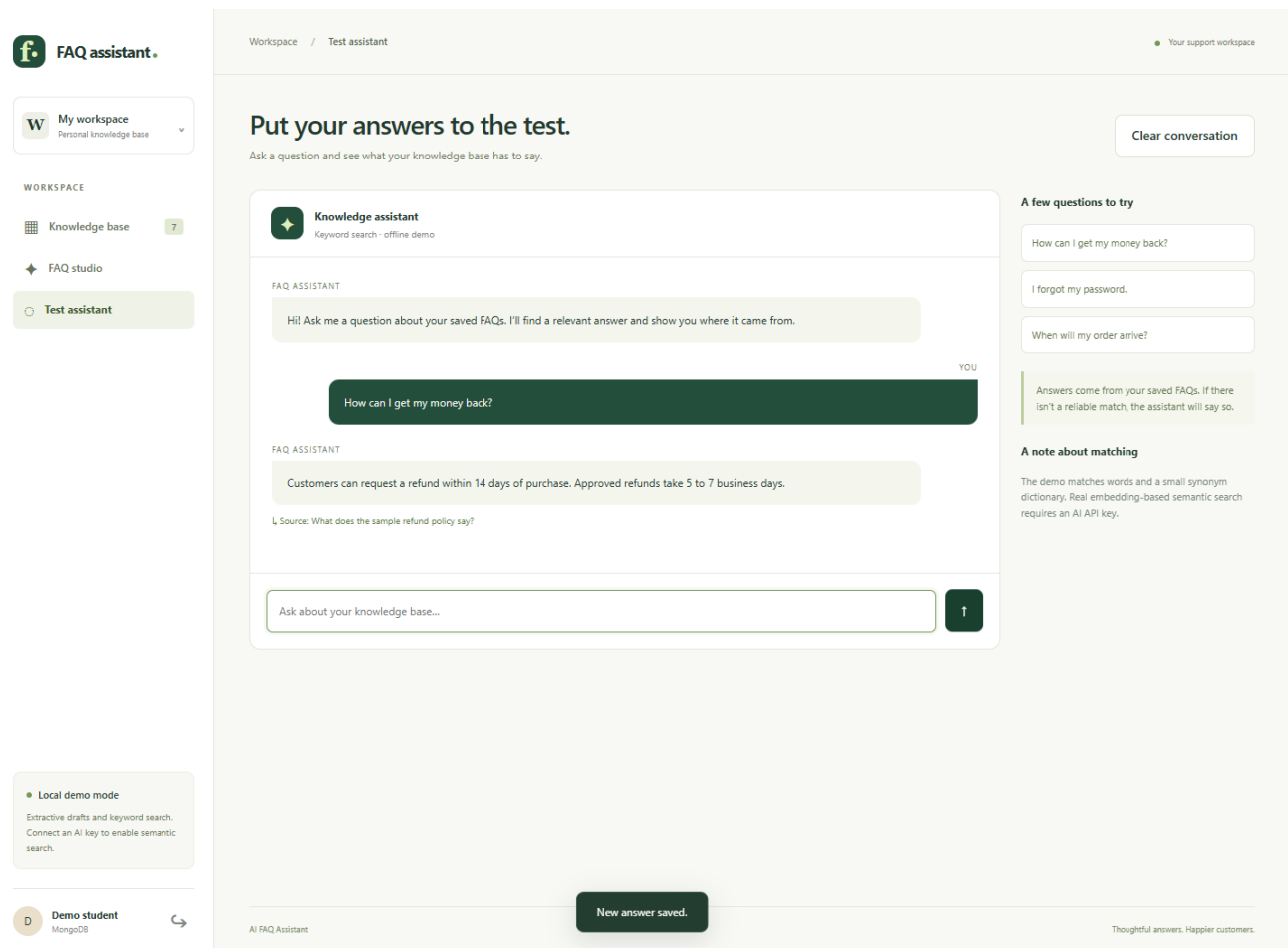
The sample screenshot shows extractive demo mode. Users can inspect a proposed question and answer, edit them, and save a draft. Live AI drafting is enabled only when a provider key is configured.

Generated drafts are not published automatically. Every saved answer can be edited or deleted from the knowledge base.

# Answer testing

The assistant retrieves an answer from saved FAQs and identifies its source.

**Figure 3. Source-backed retrieval**



The query "How can I get my money back?" retrieves the saved refund answer. The visible source allows the user to inspect the underlying FAQ. The no-key mode uses keyword and synonym matching.

An unrelated query returns a no-match message. The system does not claim that similarity scores are calibrated probabilities of correctness.

# Demonstration and references

## Run the project

1. Open `START-MONGODB.cmd` to run the application with a real local MongoDB process, or `START.cmd` for the local JSON fallback.
2. Visit <http://127.0.0.1:3000> and choose Open a sample workspace, or create an account for a reusable login.
3. Create and edit an FAQ. Open FAQ Studio, load sample content, generate drafts, then review and save one.
4. Open Test assistant, ask a question, and identify the source. Test an unrelated question and show the no-match response.
5. Add your own `OPENAI_API_KEY` in `.env` and restart only if you want a live AI demonstration. Keep keys out of the submission package.

## References

Express documentation

<https://expressjs.com/>

Mongoose documentation

<https://mongoosejs.com/docs/>

MongoDB documentation

<https://www.mongodb.com/docs/>

JSON Web Token standard, RFC 7519

<https://www.rfc-editor.org/rfc/rfc7519>

OpenAI structured outputs

<https://developers.openai.com/api/docs/guides/structured-outputs>

OpenAI embeddings

<https://developers.openai.com/api/docs/guides/embeddings>

Submission materials also include the complete source code, a Postman collection, the project report source, a presentation outline, viva preparation, and desktop/mobile screenshots.